МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Специальность: Системное и Прикладное Программное Обеспечение

Дисциплина: Системы Искусственного Интеллекта

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

“Разработка системы поддержки принятия решения на основе базы знаний или онтологии”

Игра “Ticket To Ride”

Студент: Зубулина Юлия Максимовна (465987)
Группа: P3312
Преподаватель: Болдырева Елена Александровна (157150)

г. Санкт-Петербург, 2026

Содержание:

Введение.....3
Цель работы:.....3
1. Анализ требований..... 3
1.1 Определение основных требований к системе поддержки принятия решений:..... 3
1.2 Требования к базе знаний для представления знаний:.....4
1.3 Требования к пользовательскому вводу и результатам работы.....4
2. Изучение основных концепций и инструментов..... 6
2.1. Основные концепции систем поддержки принятия решений..... 6
2.2. Использование языка Prolog..... 6
2.3. Используемые инструменты разработки..... 7
3. Реализация системы поддержки принятия решений..... 8
3.1. Разработка алгоритма работы системы..... 8
3.2. Реализация обработки пользовательского ввода..... 9
3.3. Реализация логического вывода и формирования рекомендаций.... 11
4. Оценка и интерпретация результатов..... 12
4.1. Результаты тестирования системы..... 12
Тест 1. Выбор белого цвета для игрока yuliya..... 12
Тест 2. Выбор оранжевого цвета для игрока dasha.....13
Тест 3. Поиск всех доступных перегонов для игрока dasha..... 13
Тест 4. Отсутствие подходящих перегонов.....14
Тест 5. Обработка неизвестного игрока..... 15
4.2. Оценка соответствия системы поставленным требованиям..... 15
4.3. Использование базы знаний и отличие от онтологического представления..... 16
4.4. Возможности дальнейшего развития системы..... 17
5. Заключение.....18

Введение

Цель работы:

Разработать систему поддержки принятия решений на языке Prolog на основе базы знаний, созданной в лабораторной работе № 1, и изучить принципы использования логического вывода для формирования рекомендаций пользователю. В качестве предметной области выбрана настольная игра Ticket to Ride. На основе существующей базы знаний разработана программа, которая взаимодействует с пользователем, получает его предпочтения, выполняет логические запросы и предлагает возможные действия в игровой ситуации

В рамках работы необходимо реализовать обработку пользовательского ввода, проверку корректности введенных данных, использование фактов и правил базы знаний для поиска подходящих перегонов, а также вывод рекомендаций с пояснением причин выбора. Дополнительно необходимо провести тестирование программы и оценить соответствие разработанной системы поставленным требованиям

1. Анализ требований

1.1 Определение основных требований к системе поддержки принятия решений:

Разрабатываемая система должна предоставлять пользователю возможность получить рекомендации по строительству перегонов в игре Ticket to Ride на основании текущего состояния базы знаний и введенных предпочтений

Основными требованиями к системе являются:

1. взаимодействие с пользователем посредством текстового диалога
2. получение имени игрока и желаемого цвета перегона в одной строке
3. разбор введенной строки и извлечение необходимых значений
4. проверка существования указанного игрока и допустимости выбранного цвета
5. использование базы знаний из лабораторной работы № 1 без необходимости повторного описания игровой предметной области
6. применение существующих правил Prolog для определения перегонов, доступных для строительства

7. учет цвета и длины перегона, количества карт соответствующего цвета у игрока и занятости перегона
8. формирование списка рекомендаций на основании результатов логического вывода
9. вывод пояснений к найденным вариантам
10. обработка некорректного ввода и повторный запрос данных
11. сообщение об отсутствии подходящих вариантов, если рекомендации не найдены

1.2 Требования к базе знаний для представления знаний:

Для работы системы используется база знаний Ticket to Ride, разработанная в лабораторной работе № 1. Она содержит факты об игроках, городах, перегонах, цветах, длине перегонов, количестве игровых карт, маршрутах и занятых участках игровой карты

Для формирования рекомендаций используются существующие отношения и правила Prolog. В частности, правило `can_build_segment` позволяет определить, может ли конкретный игрок построить определенный перегон. Оно учитывает характеристики перегона, наличие у игрока достаточного количества карт соответствующего цвета и доступность перегона

Дополнительно реализуется правило `recommended_segment`, которое связывает введенное пользователем предпочтение с результатами проверки возможности строительства. При выборе конкретного цвета система отбирает подходящие перегоны указанного цвета, а при выборе значения `any` рассматривает все доступные для строительства варианты

1.3 Требования к пользовательскому вводу и результатам работы

Пользователь должен иметь возможность ввести имя игрока и предпочтение по цвету перегона одной строкой. Например: `Игрок dasha, желаемый цвет any`

Программа должна разобрать введенную строку, определить игрока и желаемый цвет, а затем проверить корректность полученных значений по базе знаний. Допускается использование значений `yuliya` и `dasha` в

качестве имен игроков, а также цветов `red`, `blue`, `green`, `yellow`, `orange`, `white` и специального значения `any`

Если введенные данные корректны, программа выполняет логический запрос и выводит найденные рекомендации с указанием названия перегона, его цвета, длины и количества карт у игрока. Если подходящих перегонов нет, система сообщает об отсутствии рекомендаций. Если строка введена неверно или указаны неизвестные данные, программа выводит сообщение об ошибке и предлагает повторить ввод

2. Изучение основных концепций и инструментов

2.1. Основные концепции систем поддержки принятия решений

Система поддержки принятия решений представляет собой программную систему, которая использует имеющиеся данные и заданные правила для выбора подходящих вариантов действий. В данной лабораторной работе такая система применяется для помощи игроку в настольной игре Ticket to Ride

Основой системы является база знаний, созданная в лабораторной работе № 1. Она содержит факты, описывающие состояние игровой модели, и правила, позволяющие получать новые сведения на основе этих фактов. Благодаря этому программа может не только хранить информацию об игроках и перегонах, но и определять, какие действия доступны в конкретной ситуации

В разработанной системе решение формируется на основе имени игрока и его предпочтения по цвету перегона. После получения этих данных программа обращается к базе знаний, проверяет возможность строительства и формирует список подходящих вариантов. Таким образом, рекомендации определяются не случайным образом, а на основе заданных условий и результатов логического вывода

2.2. Использование языка Prolog

Для реализации системы выбран язык логического программирования Prolog. Он предназначен для представления знаний в виде фактов и правил, а также для выполнения запросов с использованием механизма логического вывода

Факты описывают известные сведения о предметной области. Например, предикат `player(yuliya)` указывает на наличие игрока yuliya в базе знаний, а `segment_color(moscow_smolensk, orange)` задает цвет перегона `moscow_smolensk`

Правила позволяют определять новые сведения на основе существующих фактов. В разработанной системе используется правило `can_build_segment`, которое проверяет возможность строительства перегона конкретным игроком. Для этого учитываются цвет перегона, его длина, количество карт соответствующего цвета у игрока и отсутствие занятости перегона

Для получения рекомендаций используется правило `recommended_segment`. Оно позволяет отобрать перегоны, которые игрок может построить и которые соответствуют введенному предпочтению по цвету. При выборе значения `any` система рассматривает все перегоны, доступные для строительства данным игроком

Использование Prolog позволяет сформировать рекомендации непосредственно на основе логических условий базы знаний, без необходимости вручную перечислять все возможные игровые ситуации в программе

2.3. Используемые инструменты разработки

Для реализации и проверки программы использована онлайн-среда SWISH, предназначенная для работы с языком SWI-Prolog. Она позволяет создавать и выполнять программы Prolog в браузере, задавать запросы к базе знаний и наблюдать результаты их выполнения

В качестве основы использована база знаний из лабораторной работы № 1. Новая функциональность добавлена в виде предикатов, обеспечивающих диалог с пользователем, обработку введенной строки, проверку данных, выбор подходящих перегонов и вывод рекомендаций

Для взаимодействия с пользователем применяются средства чтения текстовой строки, обработки строковых значений и вывода сообщений. Предикаты `read_line_to_string`, `normalize_space` и `split_string` используются для получения и разбора пользовательского ввода. Предикаты `findall` и `forall` применяются для сбора найденных рекомендаций и последовательного вывода результатов

3. Реализация системы поддержки принятия решений

3.1. Разработка алгоритма работы системы

Разработанная система поддержки принятия решений предназначена для формирования рекомендаций по строительству перегонов в настольной игре Ticket to Ride. Программа использует базу знаний из лабораторной работы № 1 и позволяет пользователю получить подходящие варианты на основании выбранного игрока и предпочтительного цвета перегона

Работа программы начинается с запуска предиката `recommendation_dialog_line`, который выводит приветственное сообщение и предлагает пользователю ввести данные одной строкой. В качестве примера используется строка «Игрок dasha, желаемый цвет any»

После получения строки программа выполняет ее обработку и извлекает два значения: имя игрока и желаемый цвет. Затем выполняется проверка корректности введенных данных. Имя игрока должно соответствовать одному из игроков, описанных в базе знаний, а цвет должен быть одним из допустимых значений. Специальное значение `any` позволяет получить рекомендации без ограничения по цвету

Если данные корректны, программа выполняет поиск подходящих перегонов с использованием правил Prolog. Если данные не соответствуют установленным требованиям, выводится сообщение об ошибке и запрашивается повторный ввод

После завершения поиска программа выводит результаты. Для каждого найденного перегона указываются его название, цвет, длина и количество карт соответствующего цвета у игрока. Если подходящих вариантов нет, пользователь получает соответствующее сообщение

Алгоритм работы системы включает следующие этапы: ввод данных, обработку строки, проверку корректности значений, выполнение логического запроса, формирование списка рекомендаций и вывод результата

3.2. Реализация обработки пользовательского ввода

Для взаимодействия с пользователем реализован предикат `recommendation_dialog_line`, который запускает диалог и последовательно вызывает необходимые предикаты обработки данных и формирования рекомендаций

Чтение пользовательской строки выполняется с помощью предиката `read_line_to_string`. Для нормализации введенного текста используется `normalize_space`, который удаляет лишние пробелы в начале и конце строки и нормализует последовательности пробельных символов. Предикат `string_lower` позволяет привести текст к нижнему регистру, благодаря чему регистр букв не влияет на обработку ввода

Для разделения строки на отдельные элементы используется предикат `split_string`. С его помощью программа отделяет имя игрока от предпочтения по цвету, а затем извлекает соответствующие значения из каждой части строки

После разбора выполняется проверка данных с помощью предиката `valid_preference`. Он проверяет наличие указанного игрока в базе знаний и допустимость выбранного цвета. Для значения `any` предусмотрена отдельная обработка, поскольку оно обозначает отсутствие ограничения по цвету, а не реальный цвет перегона

Если строка не соответствует ожидаемому формату или содержит неизвестные значения, программа сообщает об ошибке и повторяет запрос. Это позволяет избежать выполнения рекомендаций на основе некорректных входных данных

```
recommendation_dialog_line :-
    nl,
    writeln('Введите игрока и желаемый цвет одной строкой'),
    writeln('Пример: Игрок yuliya, желаемый цвет white'),
    read_preference_line(Player, Color),
    nl,
    writeln('Результат рекомендации'),
    show_recommendations(Player, Color).
```

```
read_preference_line(Player, Color) :-
    read_line_to_string(user_input, Input),
    (   parse_preference_line(Input, Player, Color),
        valid_preference(Player, Color)
    ->  true
    ;   writeln('Некорректный ввод или неизвестные данные'),
        writeln('Пример: Игрок yuliya, желаемый цвет white'),
        writeln('Допустимые игроки: yuliya, dasha'),
        writeln('Допустимые цвета: red, blue, green, yellow, orange, white, any'),
        read_preference_line(Player, Color)
    ).
```

```
parse_preference_line(Input, Player, Color) :-
    normalize_space(string(CleanInput), Input),
    string_lower(CleanInput, LowerInput),
    split_string(LowerInput, ",", " ", [PlayerPart, ColorPart]),

    split_string(PlayerPart, " ", " ", ["игрок", PlayerText]),

    split_string(ColorPart, " ", " ", ColorWords),
    member(ColorWords, [
        ["желаемый", "цвет", ColorText]
    ]),

    atom_string(Player, PlayerText),

    (   member(ColorText, ["any", "любой"])
    ->  Color = any
    ;   atom_string(Color, ColorText)
    ).
```

```
valid_preference(Player, Color) :-
    player(Player),
    (   Color = any
    ;   segment_color(_, Color)
    ).
```

3.3. Реализация логического вывода и формирования рекомендаций

Основная логика принятия решений реализована с использованием предикатов `recommended_segment`, `can_build_segment` и `show_recommendations`

Предикат `can_build_segment`, созданный в лабораторной работе № 1, проверяет возможность строительства конкретного перегона. Для успешного выполнения должны одновременно соблюдаться следующие условия: перегон существует в базе знаний, для него задан цвет и длина, у игрока имеется достаточное количество карт соответствующего цвета, а сам перегон не занят другим игроком

Предикат `recommended_segment` использует результат проверки возможности строительства и учитывает предпочтение пользователя. Если указан конкретный цвет, в рекомендации включаются только свободные перегоны этого цвета, которые игрок может построить. Если выбрано значение `any`, система рассматривает все перегоны, удовлетворяющие условиям строительства

Для сбора подходящих вариантов используется предикат `findall`, который формирует список всех найденных перегонов. Предикат `show_recommendations` проверяет полученный список и определяет дальнейшие действия программы. Если список пуст, выводится сообщение об отсутствии подходящих перегонов. В противном случае система последовательно выводит найденные варианты с помощью `forall` и предиката `print_recommendation`

Предикат `print_recommendation` получает из базы знаний цвет и длину перегона, а также количество карт соответствующего цвета у игрока. На основании этих данных программа выводит характеристики перегона и поясняет, почему он рекомендован: перегон свободен, а количества карт достаточно для строительства

4. Оценка и интерпретация результатов

4.1. Результаты тестирования системы

Для проверки разработанной системы поддержки принятия решений было проведено тестирование с различными входными данными. Проверялись корректность обработки пользовательской строки, определение доступных для строительства перегонов, учет предпочтительного цвета, обработка некорректного ввода и формирование рекомендаций на основе базы знаний

В ходе тестирования использовался предикат `recommendation_dialog_line`, запускающий взаимодействие с пользователем. Для проверки работы системы вводились различные имена игроков и значения предпочтительного цвета

```
% Разбор строки на игрока и желаемый цвет
parse_preference_line(Input, Player, Color) :-
    normalize_space(string(CleanInput), Input),
    string_lower(CleanInput, LowerInput),
    split_string(LowerInput, ",", " ", [PlayerPart, ColorPart]),

    split_string(PlayerPart, " ", " ", ["игрок", PlayerText]),

    split_string(ColorPart, " ", " ", ColorWords),
    member(ColorWords, [
        ["желаемый", "цвет", ColorText]
    ]),

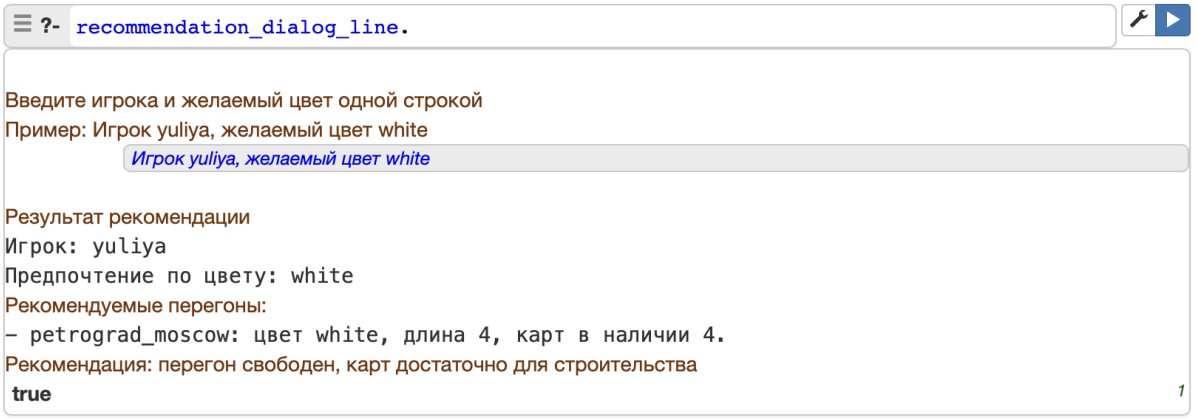
    atom_string(Player, PlayerText),

    (   member(ColorText, ["any", "любой"])
    ->  Color = any
    ;   atom_string(Color, ColorText)
    ).
```

Тест 1. Выбор белого цвета для игрока yuliya

В качестве входных данных была введена строка «Игрок yuliya, желаемый цвет white». Система успешно обработала ввод и выполнила поиск перегонов белого цвета, доступных для строительства игроком yuliya

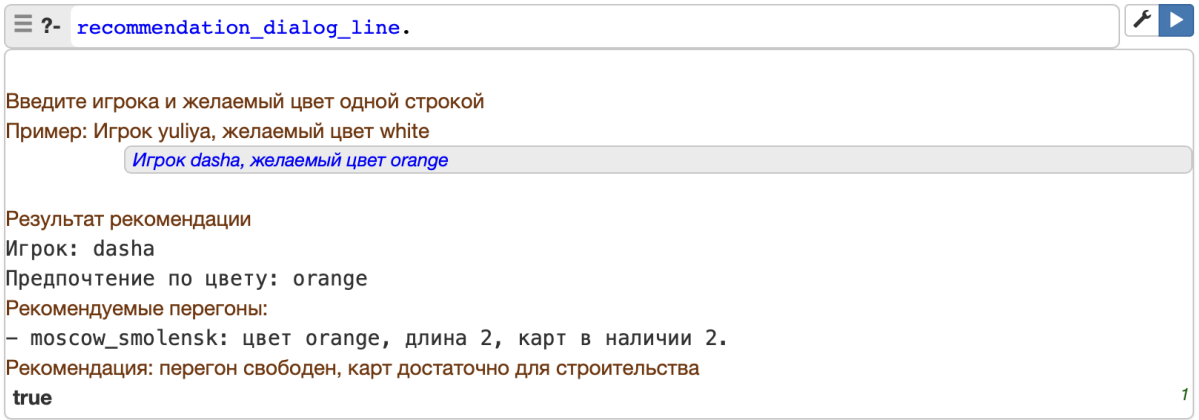
В результате был найден перегон `petrograd_moscow` длиной 4. У игрока yuliya имеется 4 карты белого цвета, а перегон свободен. Следовательно, условия правила `can_build_segment` выполняются, и система рекомендует данный перегон



Тест 2. Выбор оранжевого цвета для игрока dasha

В качестве входных данных была введена строка «Игрок dasha, желаемый цвет orange». Система определила, что игрок dasha может построить перегон `moscow_smolensk`

Длина данного перегона составляет 2, и у игрока имеется 2 карты оранжевого цвета. Перегон свободен, поэтому система включила его в список рекомендаций



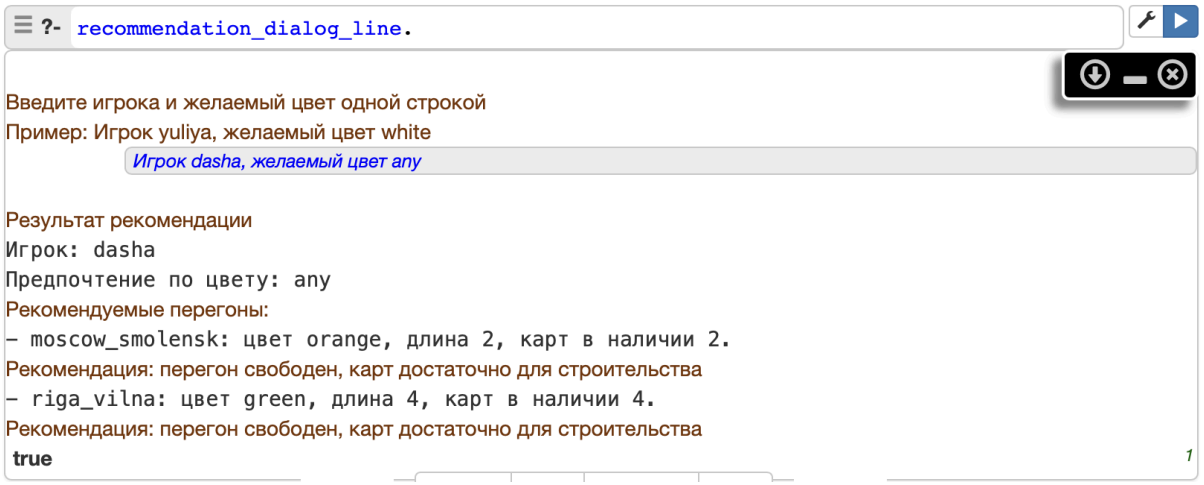
Тест 3. Поиск всех доступных перегонов для игрока dasha

Для проверки поиска без ограничения по цвету была введена строка «Игрок dasha, желаемый цвет any». Система корректно обработала

специальное значение `any` и выполнила поиск всех перегонов, доступных для строительства данным игроком

В результате были найдены перегоны `moscow_smolensk` и `riga_vilna`. Для первого перегона у игрока имеется необходимое количество оранжевых карт, для второго - необходимое количество зеленых карт. Оба перегона свободны и удовлетворяют условиям правила `can_build_segment`

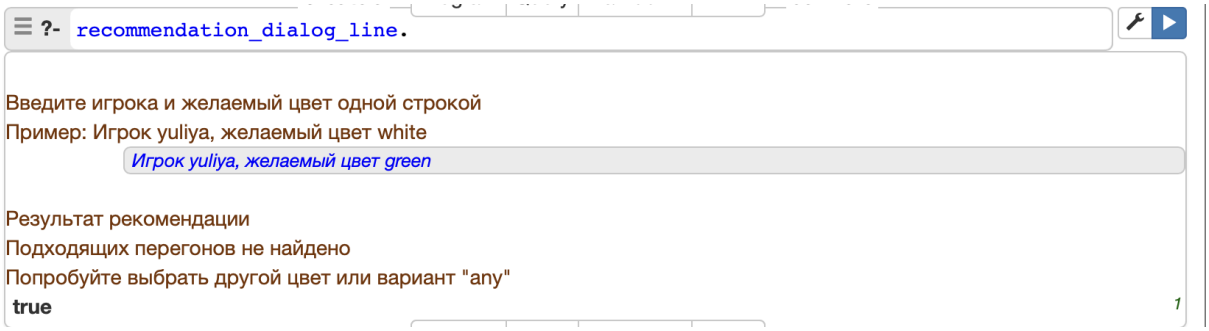
Данный тест подтвердил возможность получения нескольких рекомендаций за один запуск программы



Тест 4. Отсутствие подходящих перегонов

Для проверки ситуации, когда предпочтение пользователя не соответствует доступным вариантам, была введена строка «Игрок yuliya, желаемый цвет green»

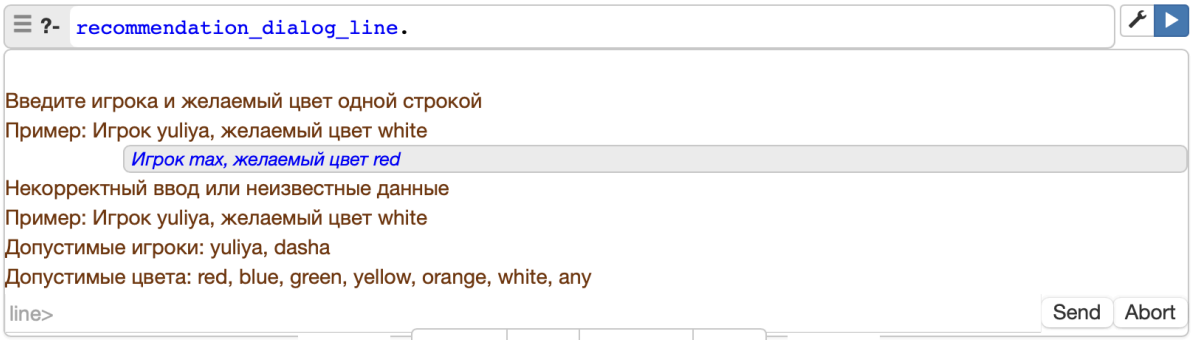
В базе знаний отсутствует свободный зеленый перегон, который игрок yuliya мог бы построить с учетом имеющихся карт



Тест 5. Обработка неизвестного игрока

Для проверки обработки некорректных данных была введена строка с именем игрока `max`, отсутствующим в базе знаний

Система обнаружила, что указанное имя не соответствует известным игрокам, вывела сообщение об ошибке и повторно запросила ввод. Рекомендации для неизвестного игрока не формировались



По результатам проведенного тестирования все пять проверенных сценариев завершились ожидаемым результатом. Система корректно обрабатывает пользовательский ввод, использует правила базы знаний для поиска подходящих перегонов и формирует рекомендации в зависимости от введенных данных

4.2. Оценка соответствия системы поставленным требованиям

Разработанная система соответствует основным требованиям, определенным на этапе анализа. Пользователь может передать имя игрока и желаемый цвет перегона одной строкой. Программа разбирает введенные данные, проверяет их корректность и использует полученные значения для выполнения логических запросов к базе знаний

Для формирования рекомендаций применяется существующее правило `can_build_segment`, которое учитывает наличие необходимых игровых карт и занятость перегона. Благодаря этому система не рекомендует строительство перегонов, для которых условия доступности не выполняются

Дополнительное правило `recommended_segment` позволяет учитывать предпочтение пользователя. При выборе конкретного цвета поиск

ограничивается перегонами данного цвета, а значение `any` позволяет получить все доступные варианты для выбранного игрока

Также реализована обработка ошибочного ввода. Если имя игрока неизвестно или строка не соответствует предусмотренному формату, программа сообщает об ошибке и предлагает повторить ввод. Если подходящие варианты отсутствуют, пользователь получает соответствующее сообщение

Таким образом, система реализует основные функции поддержки принятия решений: принимает пользовательские данные, проверяет их, выполняет логический вывод на основе базы знаний и представляет результат в форме понятных рекомендаций

4.3. Использование базы знаний и отличие от онтологического представления

В данной лабораторной работе для формирования рекомендаций непосредственно используется база знаний Prolog, разработанная в лабораторной работе № 1. Она содержит факты об игроках, перегонах, цветах, длине перегонов, количестве игровых карт и занятости игровых участков, а также правила для определения доступных действий

Логическое представление знаний в Prolog позволяет задавать условия в виде правил и получать решения посредством выполнения запросов. Например, правило `can_build_segment` позволяет проверить, может ли определенный игрок построить конкретный перегон. На основе этого правила система поддержки принятия решений формирует список рекомендаций

Онтология, созданная в лабораторной работе № 1 в среде Protege, представляет ту же предметную область с помощью классов, экземпляров и свойств, описывающих объекты и связи между ними. Такое представление удобно для структурирования знаний и визуального изучения отношений между объектами

В текущей реализации системы поддержки принятия решений запросы выполняются непосредственно к базе знаний Prolog. Автоматическое получение данных из онтологии Protege в программу не реализовано. Поэтому сравнение двух подходов в данной работе проводится на уровне

способов представления знаний: Prolog используется для логического вывода и формирования рекомендаций, а онтология - для структурированного описания предметной области и ее связей

Оба подхода могут дополнять друг друга. В дальнейшем возможно расширение системы за счет автоматического обмена данными между онтологией и программой Prolog, что позволит использовать онтологическое представление в процессе принятия решений

4.4. Возможности дальнейшего развития системы

Разработанная система может быть расширена за счет увеличения количества игроков, городов, перегонов и маршрутов, а также добавления новых игровых характеристик. Это позволит применять систему к более сложным игровым ситуациям

Одним из направлений развития является учет дополнительных факторов при формировании рекомендаций, например количества очков за строительство, близости перегона к завершению маршрута и приоритетности выполнения маршрутных карт. В этом случае система сможет не только определять доступные для строительства перегоны, но и сравнивать их полезность для достижения игровой цели

Также возможно создание графического интерфейса, автоматическое обновление состояния базы знаний после выполнения игровых действий и интеграция с онтологией Protege. Такие улучшения позволят сделать систему более удобной и расширить возможности ее применения

Выполненное тестирование подтвердило корректность работы реализованных функций в проверенных сценариях. Поставленная цель разработки системы поддержки принятия решений на основе базы знаний Prolog достигнута

5. Заключение

В ходе лабораторной работы я разработала программу на Prolog, которая помогает выбирать подходящие участки маршрута в игре Ticket to Ride. При выборе учитываются цвет участка, количество карт у игрока и доступность участка для строительства

Во время тестирования я проверила рекомендации для разных игроков и цветов, поиск всех доступных вариантов без ограничения по цвету, а также ситуацию, когда подходящих участков нет. Во время тестирования я проверила рекомендации для разных игроков и цветов, поиск всех доступных вариантов без ограничения по цвету, ситуацию, когда подходящих участков нет, а также обработку неверного имени игрока. Все пять проверенных сценариев завершились ожидаемым результатом

Поставленная цель работы достигнута. Разработанная система демонстрирует возможность использования базы знаний для поддержки принятия решений и может быть расширена за счет добавления новых игровых ситуаций, критериев оценки перегонов, дополнительных правил и интеграции с онтологией